Технологии глубокого обучения и нейронных сетей

Пара 2: как устроен и как обучается трансформер

Петр Георгиевич Меньших · МТУСИ

Модель — семейство распределений.

Обучение — выбор члена семейства, при котором данные наиболее правдоподобны.

Дальше меняется только богатство семейства и масштаб данных.

Токен в вектор

ТОКЕН	1024 ЧИСЛА, ПОДОБРАНЫ ПРИ ОБУЧЕНИИ
…	…
врач	0.12 −0.80 0.33 0.05 …
ключ	−0.41 0.22 0.07 −0.63 …
король	0.58 0.14 −0.29 0.91 …
королева	0.55 0.11 −0.25 0.88 …
…	…

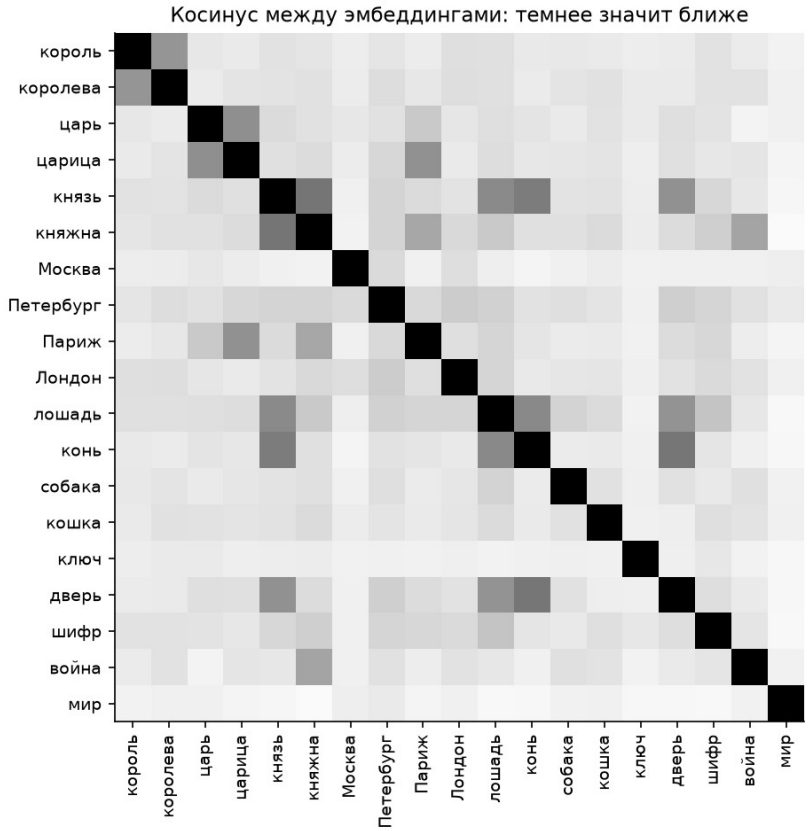
Эмбеддинг это строка таблицы.

Строк столько, сколько токенов в списке. Номер токена это номер строки.

Числа условные. Настоящие ничего не скажут глазу, у них одно полезное свойство: на следующем слайде.

Взяли строку, вот и вектор. Больше в превращении номера в вектор ничего нет.

Близкие по смыслу рядом



Близость векторов означает похожесть.

Модель не знает, что такое король. Но «король» встречается там же, где «королева», «царь» там же, где «царица», и векторы сдвинулись друг к другу.

Те же эмбеддинги, что в карте классики росли из PCA. И те же, что в RAG ищут похожие куски документов.

Косинус между строками таблицы эмбеддингов Qwen3-0.6B: темнее значит ближе. «Князь» рядом с «конем» это уже не смысл: вектор слова из кусков это среднее кусков, и совпавшие куски тоже сближают.

Одна проблема и одна метка

ключ от двери

ключ к шифру

один номер, одна строка таблицы, один вектор

Вектор токена одинаков в любом контексте.

Каждое слово должно вобрать в себя соседей.

И ОДНА МЕТКА

«собака укусила человека»
«человек укусил собаку»

Без порядка это один мешок векторов. К каждому вектору добавляется метка позиции. У современных моделей она называется RoPE.

На входе в модель у каждого токена есть вектор смысла и метка места. Дальше вопрос, как слову вобрать в себя соседей.

Библиотека без каталога

Что ищу *Query*

Запрос в голове читателя: «книга про то, как коптить рыбу».

Как меня найти *Key*

Аннотация на корешке: «рыбалка на Волге», «домашнее копчение», «история флота».

Что отдаю *Value*

Содержимое книги, то, что читатель унесет в конспект.

Читатель сравнивает запрос с каждой аннотацией и составляет конспект из всех книг, пропорционально совпадению.

В модели каждый токен одновременно и читатель, и книга.
Каждый спрашивает у всех, каждый отвечает всем.

Четыре шага

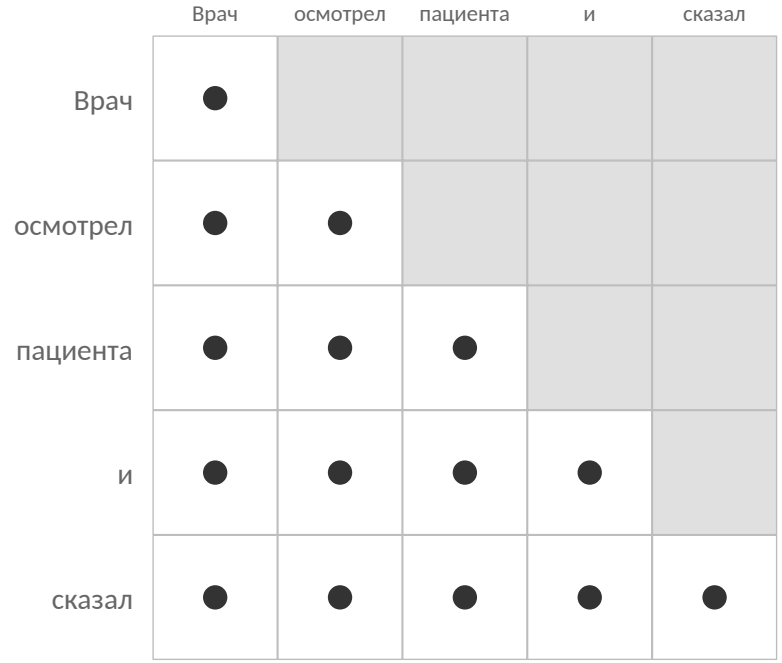
ВЕКТОР ТОКЕНА	строка таблицы плюс метка позиции
↓	
ТРИ ВЕКТОРА ТРЕМЯ МАТРИЦАМИ	запрос · ключ · значение
↓	
ОЦЕНКИ	запрос сравнивается с ключами всех токенов
↓	
ДОЛИ	softmax: тот же, что в первой паре, в сумме единица
↓	
НОВЫЙ ВЕКТОР ТОКЕНА	взвешенная сумма значений, теперь с контекстом

Почему матрицы три, а не одна

Если запрос и ключ это один вектор, каждый токен лучше всего совпадает сам с собой, и внимание сводится к «смотри на себя».

Три разные матрицы позволяют слову искать не себя, а то, что ему нужно: «его» ищет существительное слева.

Маска



серое запрещено

Токен видит только себя и то, что слева.

Модель учится предсказывать следующее слово. Если бы при обучении третье слово видело четвертое, оно бы подсмотрело ответ.

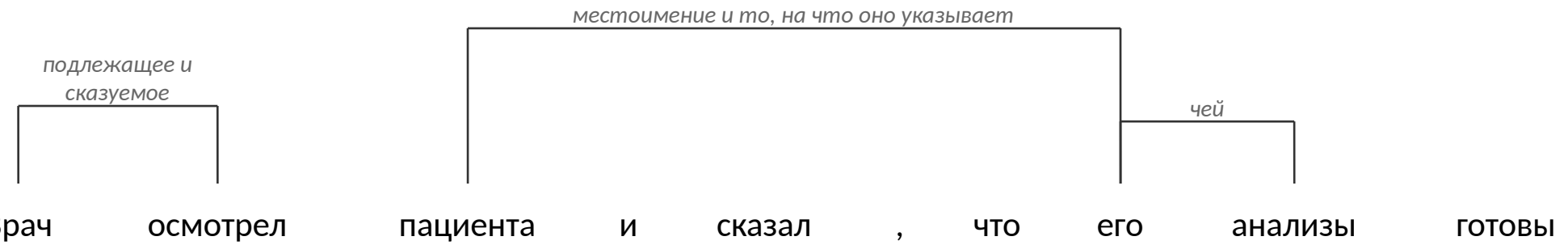
В оценки ставится минус бесконечность, после softmax эти доли ровно ноль. Модели с такой маской называются декодерами.

Из маски растет кэш: ключи и значения старых токенов не меняются, их считают один раз. Это KV-кэш.

Половина таблицы запрещена, потому что иначе учиться было бы нечему.

Головы

ОДНО ВНИМАНИЕ ЛОВИТ ОДНУ СВЯЗЬ, А СВЯЗЕЙ МНОГО



Наборов матриц несколько: у Qwen3-0.6B шестнадцать, у больших моделей до 128. Каждый набор называется головой, результаты склеиваются.

Никто не говорит голове, за чем следить, они специализируются сами.

Формула

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

$Q\,K^T$

оценки: запросы против ключей, все сразу

softmax

оценки в доли, по строке

V

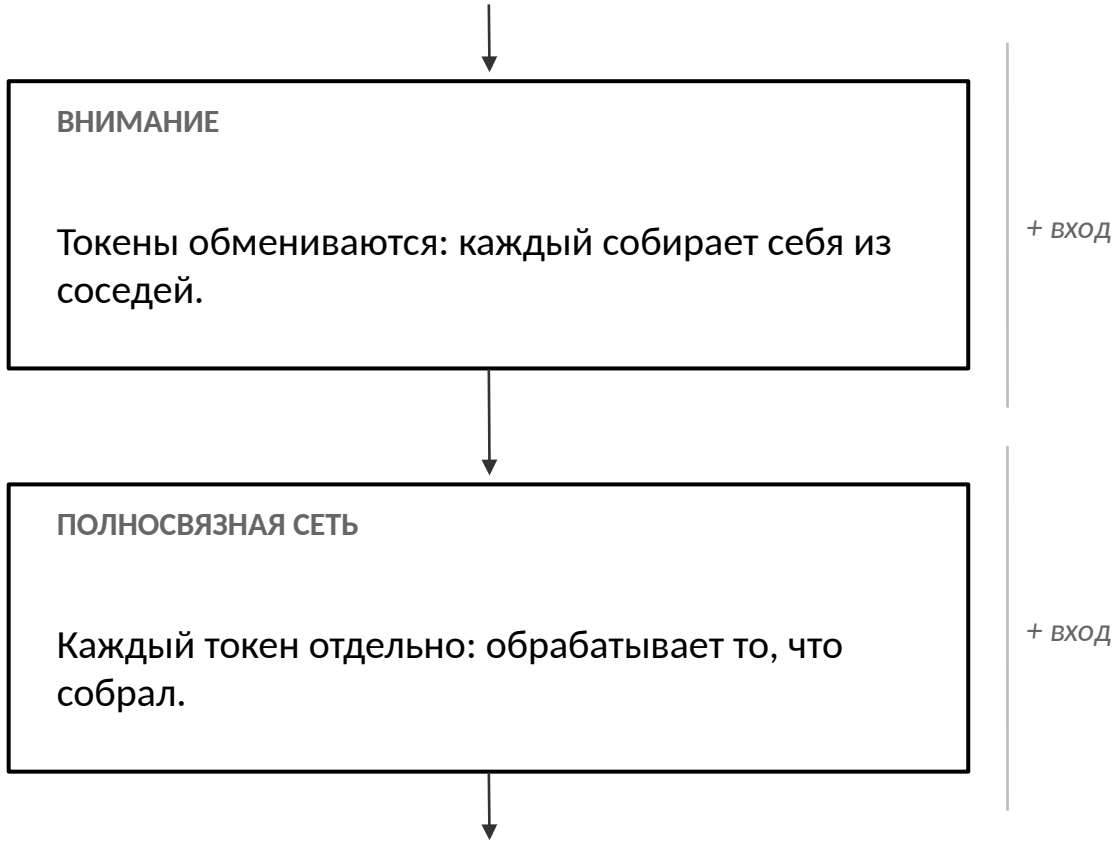
значения с долями

√d

техническая поправка, чтобы оценки не улетали

Те же четыре шага. Матрицы, размерности и корень в заметке lectures/02/why-attention.md.

Слой трансформера



Один слой это два блока подряд.

Внимание это обмен, полносвязная сеть это обработка. К выходу каждого блока прибавляется его вход, чтобы сигнал не терялся в глубине.



Стопка слоёв



У Qwen3-0.6B двадцать восемь слоев.

На каждом токен снова смотрит на соседей, но соседи уже вобрали в себя свое окружение. С каждым слоем контекст дальше и переработаннее.

Первые слои ловят грамматику, последние смысл.

Выход одного слоя это вход следующего. Токен собирает себя из соседей двадцать восемь раз подряд.

Вход и выход

текст

↓

токены

↓

эмбеддинги + позиции

↓

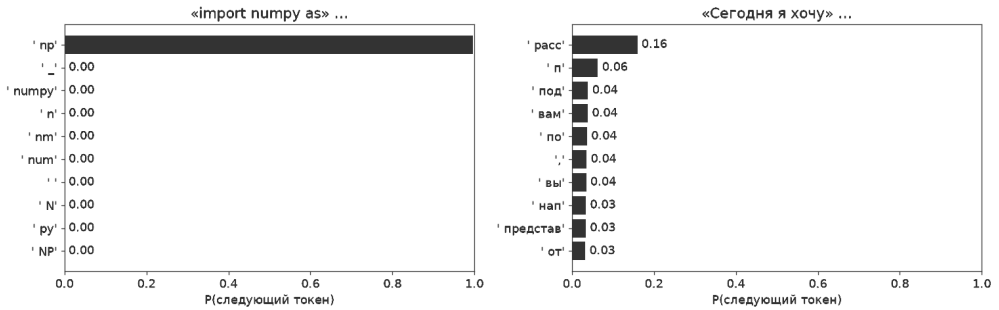
28 слоев: внимание + полносвязная сеть

↓

softmax по словарю для последнего токена

↓

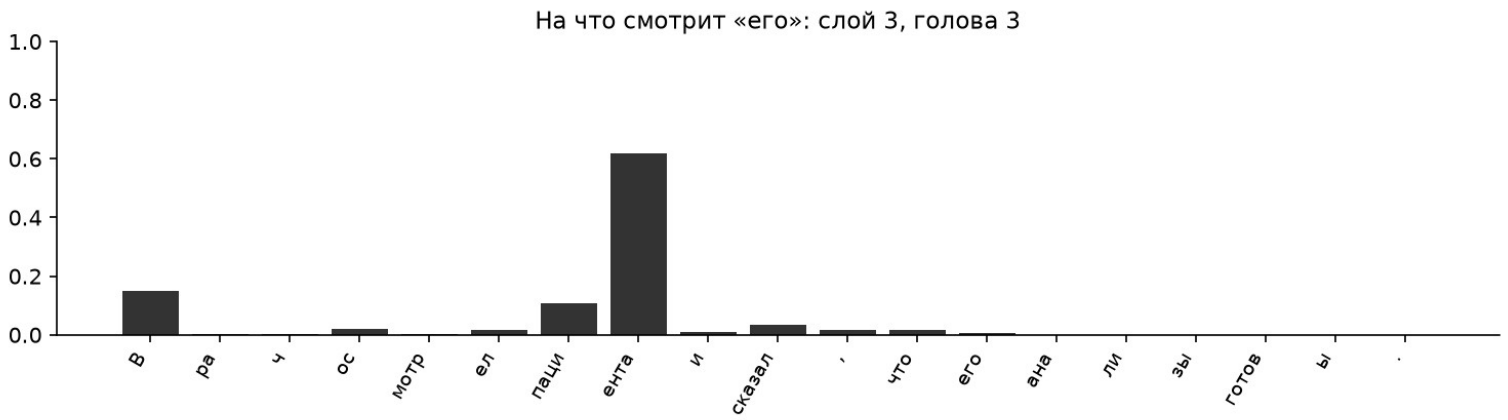
распределение следующего токена



тот же выход, что на картинке в первой паре

По сути таблица биграмм из лабораторной 1, только вместо одного предшественника весь контекст.

Внимание это не метафора, веса можно посмотреть



«Врач осмотрел пациента и сказал, что его анализы готовы»

Строка внимания токена «его» в одной из голов Qwen3-0.6B: 0.73 веса на «пациента».

Этому модель никто не учил.

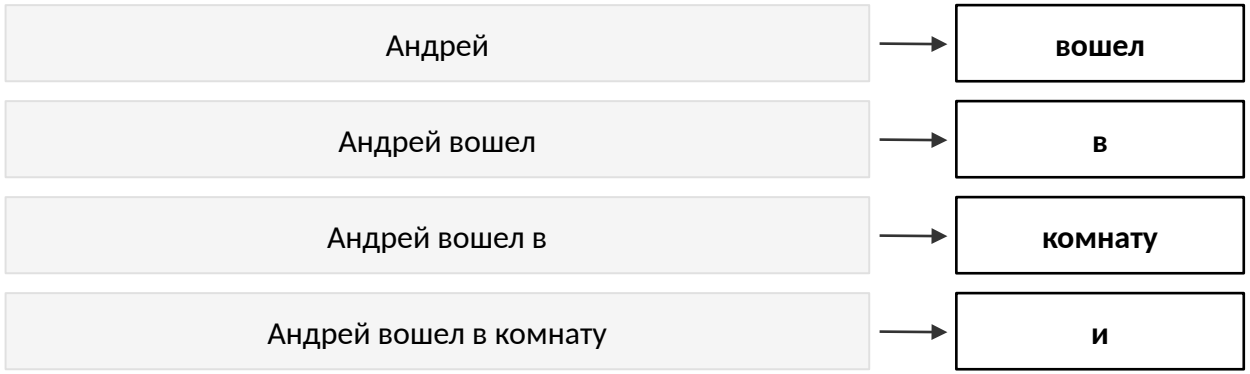
Карту внимания своей модели вы сможете посмотреть точно так же.

Задача у обучения одна: научить предсказывать следующий токен

ПОТОК ТОКЕНОВ ИЗ КОРПУСА



ИЗ КАЖДОЙ ПОЗИЦИИ ОКНА ОБУЧАЮЩИЙ ПРИМЕР



Ответ уже написан в тексте. Разметки руками нет, поэтому данных может быть МНОГО.

Та же задача, что у биграмм из лабораторной 1. Разница в двух вещах: контекст не одно слово, а тысячи токенов; и используется семейство трансформеров, а не таблица счетчиков.

Все, что модель умеет, вырастает из этой одной задачи.

Лосс

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i} \log\, p_\theta(t_{i+1} \mid t_1, \ldots, t_i)$$

p: вероятность, которую модель дала настоящему следующему токену.

Минус логарифм: чем меньше вероятность, тем больше штраф.

1/N и сумма: среднее по всем позициям корпуса.

Это уже было

Минус логарифм правдоподобия текста. Те же счетчики, только вместо таблицы 600 миллионов параметров.

Обучение это уменьшение одного числа. Формула не новая, новое только то, что стоит за p.

Числовой пример

КНЯЗЬ АНДРЕЙ ВОШЕЛ В КОМНАТУ: ИЗ СКОЛЬКИХ ВАРИАНТОВ КАК БУДТО ВЫБИРАЛА QWEN3

« Андр»	«ей»	« во»	«шел»	« в»
p = 0.02	p = 0.88	p = 0.001	p = 0.22	p = 0.84
из 50	**из 1.1**	**из 1000**	**из 4.5**	**из 1.2**

в среднем из 12 вариантов на каждом шаге

Это перплексия. Среднее геометрическое: перемножить и взять корень пятой степени.

Лосс это логарифм числа вариантов: ln 12 = 2.52. Логарифмы хорошо складываются, поэтому в формуле он, а не сами варианты.

Наугад из 151 936

Худший случай: из всего словаря на каждом шаге.

Идеальная модель

Из одного варианта. Такой не бывает, потому текст не предопределен.

Дорого стоит то, что нельзя угадать: имя после «Князь», глагол после имени. Дешево то, что доопределено.

Градиентный спуск

Пачка окон
из корпуса

→

Лосс
на пачке

→

Градиент
по всем параметрам

→

Сдвиг
против градиента

И снова, миллионы раз. Та же цепочка, что и для монетки: лосс, производная, шаг.

ЧТО МЕНЯЕТСЯ С МАСШТАБОМ

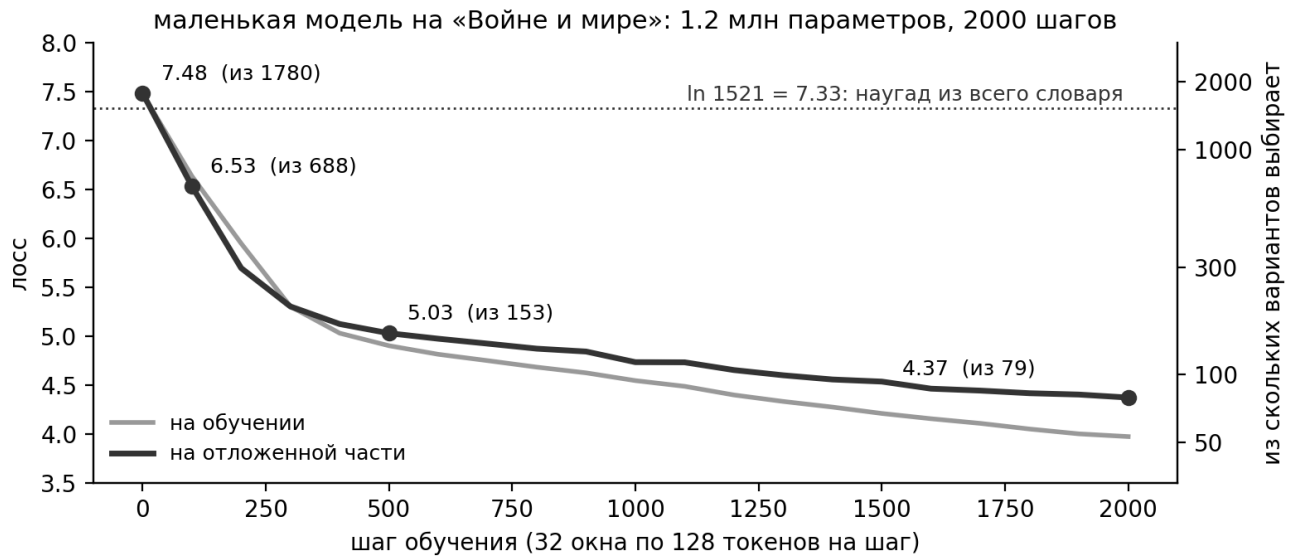
Параметров миллиарды
а не десять

Шагов миллионы
а не сто

Тысячи видеокарт одновременно
каждая со своей пачкой, раз в шаг обмен градиентами

Со времен перцептрона на уровне принципа не изменилось ничего. Изменились три числа.

Кривая обучения



ЗАТРАВКА «КНЯЗЬ АНДРЕЙ», ТЕМПЕРАТУРА 0.8

шаг 0

увиддруга ва светсебе позицмноядсами головельсмотря

шаг 100

ся и то мыслней е слышма ивон кажней про которые

шаг 500

что бы отец и придать что ж мне блаженно не хотал между тем что он чувствовал

шаг 2000

подъехал к местам бенигбочки и началый пехотный совет от линии

Модель выбирала из 1780 вариантов, теперь из 79. Все, что она умеет, появилось из уменьшения одного числа.

Цена

работа ≈ 6 × параметры × токены

прямой проход два действия на параметр на токен, обратный четыре

	параметры × токены	операций	время
Маленькая модель из работы 2	1.2 млн × 8 млн	$6 \cdot 10^{13}$	секунды арифметики
Qwen3-0.6B	0.6 млрд × 36 трлн	$1.3 \cdot 10^{23}$	порядка ста тысяч часов одной H100
Llama 3 405B	405 млрд × 15.6 трлн	$3.8 \cdot 10^{25}$	30.8 миллиона часов H100 по отчету Meta

H100 на практике выдает 3–4 · 10^{14} операций в секунду. Подстановка чисел Llama в формулу сходится с отчетом.

С нуля обучают десяток компаний в мире. Все остальные берут готовые веса и дообучают: токенов меньше на четыре-пять порядков.

Под капотом генерации



Одни и те же логиты при трех температурах: «сказал» 0.32 при 0.3, 0.07 при 1.0, 0.03 при 1.5. На десятку токенов приходится 97 %, 39 % и 19 % массы.

Три шага руками

После «Князь Андрей вошел в комнату и»: «сказал» 0.066, « с» 0.061, « у» 0.060. Модель не уверена.

После «сказал»: двоеточие 0.51.

После двоеточия: кавычка 0.55.

Три forward pass, три выбора. Внутри generate ничего другого нет.

Модель не пишет текст. Она на каждом шаге выдает распределение, а токен из него выбирает код.

Что выбирает код и что делает дообучение

ЗАТРАВКА «КНЯЗЬ АНДРЕЙ ВОШЕЛ В КОМНАТУ И», BASE-МОДЕЛЬ

Жадный выбор

…сказал: «Вот, пожалуйста, выслушайте меня. Я хочу, чтобы вы меня поняли. Я хочу, чтобы вы поняли, что я не хочу, чтобы вы меня убивали. Я хочу, чтобы вы поняли…» Цикл с 26-го токена.

Температура 0.7

…висел на письменной ложке. Он начинал с самописных имен, и все буквы в номинальном окончании были в верхнем регистре.

Температура 1.5

…вис меня на писше «его место обращ وحتى นิยม изм AYS постоя gtk Гвы все…» Хвост словаря выпал: смесь языков.

Top-p 0.9

…висел на письменных ложках. Один из них была прямой и управляла электричеством. Второй медленный и управляет нитями.

BASE на «Почему небо голубое?»

«Почему небо голубое? Может ли мое суждение „небо голубое“ означать, что я и все люди, которые видят его, имеют одну и ту же цветовую натуру? Какие существуют другие феномены…»

INSTRUCT, та же модель после дообучения

«Небо голубое потому, что солнечный свет проникает в атмосферу, а затем проходит через теплые молекулы и поглощает его…» Отвечает. Уверенно и физически неверно.

Жадный выбор зацикливается, высокая температура рассыпает текст. Base продолжает, Instruct отвечает: между ними дообучение.

Потеря требования на длинной задаче

Требование: в конце каждого ответа пиши слово КОНЕЦ

тысячи токенов: код, документы, вывод инструментов

Ответ без КОНЕЦ. Без отказа, без оговорок, буднично.

СЛУЧАЙ

Требование в начале, длинная работа, в конце требование тихо нарушено. Каждый ответ выглядит правильным.

В ЧЕМ ОШИБКА

Модель не отказалась и не ошиблась явно. Она перестала учитывать то, что было далеко.

МЕХАНИЗМ, ДВА

Размывание: доли внимания в сумме единица; инструкции из начала достается 0.021, потом 0.009, потом 0.003.

Обрезка: контекст не влез, старое отрезано или пересказано, инструкции физически нет.

ВЫВОД

Важное повторяют и держат близко к концу. Длинную задачу режут на короткие с явным состоянием.

Лабораторная 2: свой трансформер

ТЗ в репозитории курса.

Своя модель и ее обучение

Токены из вашего BPE работы 1, те же данные и отложенный текст

Декодер на PyTorch: внимание по формуле этой пары, слой, стопка

2000 шагов: кривая лосса, образцы по шагам

Около миллиона параметров, 10–20 минут на процессоре

Генерация и сравнение

Своя generate: температура, top-p, жадный выбор

Четыре режима на одной затравке, шаг зацикливания

Одни данные, одна метрика, два семейства

Полный цикл разработки и предобучения модели: от лосса на случайных весах до связных обрывков фраз.

Что дальше в мире языковых моделей

От Base к ассистенту

Дообучение на диалогах: модель отвечает, а не продолжает

Обучение на предпочтениях: отвечает так, как нравится людям, и отсюда же соглашается, когда не надо

Адаптеры: дообучить за часы, а не за месяцы

Модель в сервисе

Факты и документы кладут в контекст, а не спрашивают из памяти: retrieval

Инструменты: модель пишет запрос, код его исполняет

Тестовый набор с вопросами без ответа

Правило инженера

Модель делает узкую работу, остальное делает код

Проверяйте модель: она уверенно ошибается

Думать головой это чтение осей и поиск механизма

Подробно: лекции 4–6 очного курса в репозитории, заметки и ноутбуки открыты.

Все, что модель умеет и чего не умеет, следует из одной задачи: предсказать следующий токен правдоподобно.

Спасибо



github.com/pmenshih/tgo-course

Материалы обеих пар, задания работ, заметки для любознательных